Strata — BA Guide

For business analysts

strata-kb v1.35.0 · 2026-11-06

1. Who this guide is for

You write requirements: tickets, user stories, acceptance criteria and, for large features, mission plans that break them down. A ticket you finish is **Dev-ready**: every claim that touches a standard comes from a real document, at a known revision, and stays checkable months later when someone asks where it came from.

Your problem is not writing. It is **grounding**: Strata gives you search over published knowledge, **citations that pin a version** and a **Definition-of-Ready gate** (`kb ticket lint`, `kb mission lint`), so a ticket cannot reach a developer with a broken or missing citation. You **publish your plan** to the hub when it is ready, and **amend** a mission or ticket in place when the knowledge, architecture, scope or delivery moves.

A BA repo does	A BA repo does not
Read the hub's published knowledge	Ingest documents or summarize anything
Cite it, pinned to a hub commit	Publish knowledge content
Version tickets and missions in Git, and gate them in CI	Push to your issue tracker: the assistant never opens a ticket for you
Publish its own plan, `<repo_id>-plan`, when you publish or merge	Publish drafts, or anything you did not mark ready
Run full-auto mode when you switch it on (section 4)	Merge or ship anything by itself otherwise

Your repository is strictly **read-only** against the *knowledge* base. You cite, you never contribute. That is deliberate: it keeps requirements traceable to a knowledge version without giving the requirements process a way to change the knowledge itself. The one thing you send to the hub is your own plan: the missions and tickets you have reviewed, as one generated document. It sits next to the knowledge and never edits it.

Shared concepts, such as the L0 to L3 layers and `kb-context` citations, are explained in the architecture document.

Slash commands. Every slash command in this guide exists for Claude Code, GitHub Copilot and Cursor, with these exceptions:

- `/kb-publish` is a skill in Claude Code (`.claude/skills/kb-publish/`), not a file in `.claude/commands/`. You still type `/kb-publish`.
- A BA repo has no `/kb-ingest`, `/kb-summarize`, `/kb-init` or `/kb-l3-fix`: it never ingests or summarizes.

The Dev side of the hand-off (fetching your ticket, asking you questions, the Dev's own auto mode) is in the Developer Guide.

2. First-time setup

2.1 Install and scaffold

```
pip install strata-kb
kb init --kind ba
```

`kb init --kind ba` writes the config, the MCP wiring, the slash commands and skills for all three assistants, `.claude/settings.json` (merged into yours), `tickets/` and `missions/`, the templates under `docs/`, your `*.local.md` quality overrides (created once, never refreshed) and four workflows: `kb-ticket-lint.yml`, `kb-doctor.yml`, `kb-publish.yml` and `kb-publish-request.yml`. Re-running it is safe (6.2).

2.2 Point at the hub and register the repo

```
# .kb/config.yaml
kind: ba
hub: https://github.com/acme/kb-hub.git
repo_id: acme-requirements
intake: https://kb-intake.acme.com
```

- **`hub`** is what `kb ticket lint`, `kb query` and `kb resolve` read references from.
- **`repo_id`** names your published document, `<repo_id>-plan`, and must equal this repo's id in the hub's `federation/registry.yaml`. Ask the hub operator to add your `owner/repo` under `repos:` mapped to that id, and to leave `workflow:` unset: the intake compares it to the full `job_workflow_ref`, which includes the publish tag, so a pin never matches. Publishing is refused until you are registered. Branch protection on the hub plus the OIDC intake are the actual security boundary.
- **`intake`** is the hub's intake service URL (the operator gives it to you). With it, `kb publish` pushes a `kb-publish/<ts>` tag and CI runs `kb ci-publish`, which authenticates to the intake with GitHub Actions OIDC inside your own job and needs no secrets. Without it, `kb publish` takes the snapshot route (a direct mirror or a PR on the hub, no CI, no intake).

Never put a token in `hub:` or `intake:`. The file is committed, so the token lands in history. `kb doctor` warns; rewrite the URL, pass the credentialed URL in the `STRATA_KB_HUB` environment variable (it overrides the file), and rotate the token. Any machine that uses a credentialed hub URL needs Git 2.31 or newer.

`hub:` and `STRATA_KB_HUB_URL` are different values: `hub:` is the git URL or path of the hub repo; `STRATA_KB_HUB_URL` is the hub's HTTP base URL, used by MCP and the auto runner.

2.3 Connect your assistant

`.mcp.json` (Claude Code) and `.cursor/mcp.json` (Cursor) point at the hub through two placeholders, `${STRATA_KB_HUB_URL}` and `${STRATA_KB_HTTP_TOKEN}`. Run `kb mcp-setup` (in your assistant: `/kb-mcp-setup`). It asks for the hub's HTTP base URL and token (get the token from the hub maintainer), writes both into `.env`, makes sure git ignores that file, and verifies them against the hub, so a wrong URL and a rejected token come back as different errors. A bare re-run reads both back out of `.env` and verifies again.

Prefer the hidden prompt or the `STRATA_KB_HTTP_TOKEN` variable over `--token`. That flag leaves the token in your shell history and in the process listing that other users on the machine can read. `kb mcp-setup` also warns when the URL is plain `http://` to a non-loopback host, because the token then crosses the network in clear.

The MCP config reads those variables from your environment, not from `.env`, at the moment your assistant starts. Load the file (`set -a; source .env; set +a`, or use direnv) and restart the assistant.

One version for every repo. Every repo runs the hub's strata-kb version. Upgrade the hub first. Then, once `kb mcp-setup` has run (above), run `kb doctor` in this repo: it installs the hub's version, sets the `strata-kb==<hub version>` pins in `.github/workflows/`, and refreshes the skills and commands. In CI it never installs, re-pins or refreshes skills; a CI job that runs `kb doctor` with `STRATA_KB_HUB_URL` and `STRATA_KB_HTTP_TOKEN` set reports a version mismatch as an error instead (none of the scaffolded workflows runs it). With `STRATA_KB_HUB` or `--hub` set, `kb doctor` skips the MCP path and the version check, and only reports that the version was not checked against the hub. The files under `.github/instructions/` and `.cursor/rules/` refresh only through `kb init`. An unpinned `pip install strata-kb` in a job that holds `id-token: write` would let every run pick up whatever PyPI serves, in a job able to write to the hub; that is why the workflows are pinned.

2.4 Turn on the CI gate and the daily sweep

Setting	Where	What it is
`STRATA_KB_HUB`	Actions, **Variables**	The hub the gate resolves references against
`KB_HUB_TOKEN`	Actions, **Secrets**	A read token, only for a private hub
`KB_FAIL_ON_STALE`	**Variables**, optional	Any value makes an upstream amendment fail the gate

Then, in branch protection, **require the `lint` check** (the job in the `kb-ticket-lint` workflow) on the branch your tickets land on. Lint running in CI does not block a merge by itself; requiring the check does. The workflow runs on every pull request, not only on changes under `tickets/` or `missions/`, on purpose: a path filter on a *required* check would leave an unrelated PR waiting for a check that never starts. The job inspects the diff and exits cleanly when there is nothing to lint.

> A pull request opened **from a fork** cannot read repository secrets, so on a private hub the gate fails there with a hub-unreachable message. That is the gate refusing to go green without checking, not a network fault. Merge fork contributions through a branch in this repository.

`kb-doctor.yml` runs `kb ticket stale` over `tickets/` and `missions/` every day at 03:17 UTC and on demand, with the same variable and secret. The publish workflows need neither: they authenticate with OIDC.

---

3. Daily work

Task	Run	Result
Plan a large feature	`/ba-mission-plan`	`missions/M-<slug>.md`
Write a ticket	`/ba-ticket-author`	`tickets/<folder>/<ticket-id>.md`
Report a defect	`/ba-bug-report`	`tickets/bugs/BUG-<n>.md`
Answer a Dev's question	`/ba-answer-questions`	the ticket amended, the answer on the hub
Change a mission or ticket	`/ba-mission-plan --amend M-<slug>`, `/ba-ticket-author --amend <ticket>`	one revision, one change-log row
See which story is next	`kb mission next`	done / ready / draft / to-draft / blocked
Publish the plan	`/kb-publish`, or merge to `main` (as scaffolded)	the plan on the hub

Everything the agent does, you can do at a terminal: `kb query "<text>" [--tags t]`, `kb get <doc> <section> [--level l2]`, `kb context new --refs "doc §4.2, doc §4.3"` (paste the block under `## KB context`), `kb ticket lint <file>`, `kb mission lint <file>`.

3.1 Plan a large feature: the mission

A feature spanning several user stories gets a **mission plan** first; small work goes straight to a ticket. Run `/ba-mission-plan` and describe the need. The pipeline is **Intake, Ground, Draft, Split, Ground services, Pin, Lint, Maturity review, Review**. It saves `missions/M-<slug>.md` with a C4 Level 1 diagram (System Context), a C4 Level 2 diagram (Container), a scope split and a user-story backlog whose ids derive from the mission id.

```
kb mission lint missions/M-checkout-revamp.md
```

The gate needs the structure, both diagrams, a well-formed backlog and every citation resolving at its pinned version. A `0/N US drafted` warning is normal: the tickets do not exist yet. Whether the backlog is complete stays **your** judgement. If you drop a story, leave its number retired: renumbering would break the file names of tickets already drafted.

The SA grounds the service list (step 5, run for you): `/sa-ticket-ground --mission` fills `## Services & order`, and the gate reports FAIL on each `[NEW: D<n>]` row until you flip it to `DECIDED` (3.3). In a skeleton repo the agent stops at step 5b, **Decide**, and shows you the table once.

3.2 Write a ticket

Run `/ba-ticket-author` and describe the business need. The agent runs a nine-step pipeline; your jobs are steps 1, 3 and 9.

```
 1 Intake            you describe the need
2 Parent mission    optional, name the mission this story belongs to
3 Ground            kb_search; YOU pick which sections apply
4 Draft             story, Given/When/Then ACs, use cases, business flow
5 Pin               kb_context_new embeds the pinned block
6 Lint              kb ticket lint until DoR: PASS
7 Ground technical  the SA fills Technical grounding; kb ticket check
8 Maturity review   two independent reviews, one round by default
9 Review, save      you read it, commit it, paste it into the tracker
```

- **Ground.** The agent shows **every candidate section**, not just its favourite. Pick the ones that genuinely apply. Your domain judgement cannot be delegated here: the agent cannot know that a section is superseded in practice by a local policy.
- **Maturity review.** Two independent reviews against `docs/review-rubric.md` (Business coverage, Dev implementability). A gap it cannot close becomes an owned open question, `OPEN(<owner>)`, never a guess: read those, they are where the agent knew it did not know. Round limits are in the skill. The review scores are agent-reported: `kb ticket lint` checks only that `## Review record` has the right shape, not how a score was reached.
- **Review and save.** The draft lands in `tickets/<folder>/<ticket-id>.md`. You read it, commit it and paste `kb ticket export tickets/<folder>/<ticket-id>.md` into the tracker: the ticket without guidance comments, `## Definition of Ready` and `## Review record`.

Linking to a mission. Name the parent mission when you draft a story from its backlog. The agent writes `> Parent mission: <mission-id> @ rev <N>` under the title and saves `tickets/<mission-id>/<mission-id>-US<n>.md`. Every story the ticket's `- Blocked by:` names must also be in its `Depends on` cell in the mission's `## Sequencing`, or `kb ticket lint` fails: the Dev's auto runner reads only `Depends on`. `kb ticket tidy` moves a flat `tickets/` into the per-mission layout by the parent line.

One ticket, one target repo. A ticket is built in exactly one dev repo. The hub routes it there: a `> Target repo: <repo-id>` line in the header block (like `> Type:`, never in the body) wins; without it the target is the repo of the ticket's first qualified `Grounded on: <repo>:<repo>-code` entry, which the SA writes (an unqualified `-code` entry resolves by its unique holder on the hub). `kb ticket lint` errors when the header and a qualified grounding name different repos. With one dev repo on the hub every ticket goes to it and none of this is needed. With two or more, a ticket with no target is blocked as `no target repo`, and a target that is not a dev repo on the hub is blocked as `target <x> is not a dev repo on the hub`. Work that spans two repos is two tickets, or a parent story split into children (3.10). A `- Blocked by:` line in a ticket outside a mission blocks only on ids published in the same BA plan; tracker keys and prose never block.

3.3 The SA grounds the technical half

Step 7 runs this for you. Once lint reports `DoR: PASS`, the agent saves the draft and runs `/sa-ticket-ground` as its own subagent. It fills the SA-owned `## Technical grounding` (service, files, tables, routes, pages, jobs, symbols, test command), each a section id from the hub's `<repo>-code`, and never edits your sections. Code that does not exist yet is a design decision: the SA proposes a `## Technology decisions` row in the parent mission (status `OPEN`, a human owner) and writes `[NEW: D<n>]`. With no parent mission it writes a free-text `[NEW: <reason>]` note instead.

`kb ticket check tickets/<folder>/<ticket-id>.md` gates it (exit `0` PASS, `1` FAIL):

Output	Meaning	What to do
PASS, `[note] ... still valid`	Nothing the ticket cites changed	Nothing
PASS, `[warn]` with a diff	A cited section changed, every cited name still holds	The developer judges the diff
FAIL, `re-run /sa-ticket-ground`	A cited id, column, route, command or file is gone	Run `/sa-ticket-ground` by hand
FAIL, names an `OPEN` row and its owner	A `[NEW: D<n>]` points at an undecided decision	Flipping it to `DECIDED` is your call, never the agent's
FAIL, `Open decisions` not empty	The document cannot prove something	Yours or the Dev's to decide; no re-ground

The handover lists open rows in a `## Needs input` block. How the SA tells "not built" from "not scanned" is in 6.4.

3.4 Which story next

```
kb mission next
```

State	Means
`done`	Its id is in a `hist.*` row of the hub's `<repo>-svc`, written at the Dev's handover and published by the Dev's `kb publish` after the merge
`ready`	The ticket exists and every `## Definition of Ready` box is ticked
`draft`	The ticket exists but a box is unticked, the section is missing, or the parent mission moved on
`to-draft`	No ticket yet, every `Depends on` story is `done`, every D-row that `Blocks` it is `DECIDED`, no open question names it in `blocks:`
`blocked`	Named reasons: `US <id> not done`, `D<n> OPEN (owner: <x>)`, `Q<n> open (owner: <x>)`

The report ends with `Next: <us-id> — <title>`; `/ba-ticket-author` with no argument runs it first and proposes that story. A dependency that is only `draft` or `ready` still blocks: done means published on the hub. A ticket is also `draft`, reason `parent mission rev <M> affects this story`, when its mission's `> Revision:` is above the ticket's `@ rev` pin **and** a later change-log row lists the story in `Affected`. The command is read-only and exits `0` except on a usage error. `--kb-dir` is read first when it holds the `-svc` document (a dev machine), the hub second; an unreachable hub or unpublished `-svc` becomes a `note:` line. With `--from-hub` (and `--repo <ba-repo-id>`) it reads the published plan instead: a published ticket is `ready`, and `draft` cannot be seen.

3.5 Report a bug

A defect in something live is a **bug ticket**: no "As a ... I want ...", and its acceptance is "the steps no longer reproduce the wrong result". Run `/ba-bug-report`. It asks what broke, the environment, the exact steps, expected and actual result, severity and screenshots, drafts `tickets/bugs/BUG-<n>.md` from `docs/tickets/BUG-TEMPLATE.md` and lints it until `DoR: PASS`. A bug skips the maturity review and the technical grounding.

- **Header lines.** `> Type: bug|hotfix` (a hotfix ships ahead of the next release) and `> Severity: S1..S4`, both required; optionally `> Related ticket: <id|none>`. No `> Parent mission:`. The agent also asks which dev repo the fix lands in and writes `> Target repo: <repo-id>` (you choose; with only one dev repo on the hub you may leave the line out, since the hub routes the bug to it). Without a target, a hub with two or more dev repos blocks the bug as `no target repo`.
- **Screenshots.** PNG or WebP only, in `tickets/bugs/BUG-<n>/`, referenced as `![caption](BUG-<n>/<file>.png)` under `## Attachments`; with none, `N/A — <reason>`.
- **Your part.** Tick the Definition of Ready after reading the draft (the agent never does), commit the bug with its folder and publish. The Dev then runs `/dev-fix-bug BUG-<n>`.

3.6 Answer a Dev's questions

While designing or planning a ticket, a Dev may ask a business question that decides what an acceptance criterion means (the Dev side). The questions live on the hub. Run `/ba-answer-questions [ticket]`: it lists this repo's open questions and walks them one ticket at a time, oldest first, each with its context, the ACs it blocks and who asked. The answers are yours; the agent never writes one for you. It needs MCP pointed at the hub's HTTP server (`kb mcp-setup`); there is no CLI for it.

- **Format.** Questions and answers use fixed English labels (`**Decision needed:**`, `**Decision:**`, `**Ticket changes (rev N):**`, and so on); the skill writes them and the content follows the ticket's language.
- **One amendment per ticket.** An answer that changes behaviour amends the ticket as in 3.7: `> Revision:` becomes N+1, one `## Change log` row with Trigger `question`, What changed `Q<ids>: <one line>`, one maturity review round, and lint passes. All the questions of a ticket go into the same amendment.
- **Re-tick the Definition of Ready.** The amendment unticks the boxes whose evidence changed, and `kb plan-ingest` drops a ticket whose boxes are not all ticked, which would take it off the hub. Until every box is ticked the skill records no answer for it.
- **Waiting for publish.** The skill never commits, pushes or publishes. Until revision N+1 reaches the hub the question shows as *waiting for publish* and the Dev is told not to act on it; an answer that changes nothing is recorded as answered at once. The hub's **Q&A** menu (`/ui/questions`) shows open, waiting and answered questions.

3.7 Keep missions and tickets current

Every mission and ticket carries `> Revision: <N>` under its title, starting at 1, and an append-only `## Change log` before `## Review record`:

```
| Rev | Date | Trigger | What changed | Affected | Review |
|---|---|---|---|---|---|
| 2 | 2026-10-14 | kb-context, scope | Refund window cut to 14 days | M-checkout-US3, M-checkout-US5 |
```

`Trigger` is one or more of `kb-context`, `architecture`, `scope`, `delivery`; `question` is added only by `/ba-answer-questions`. In a mission, `Affected` lists the **full** story ids (`M-<slug>-US<n>`; a bare `US3` is not matched). Lint needs exactly one row per revision `2..N`, in order, with valid triggers.

```
/ba-mission-plan --amend M-checkout
/ba-ticket-author --amend M-checkout-US3
```

Neither starts from Intake or renames anything. The agent asks you for the trigger and what changed (drift findings pre-fill the answer, it never guesses), applies the change rules, bumps `Revision`, appends the row, lints, runs one maturity review of the changed sections and hands you the diff. It never commits or pushes. Amending a ticket **unticks** the Definition-of-Ready boxes whose evidence changed.

Who ticks the Definition of Ready. The agent ticks, naming the evidence, every box a tool proves after its last edit (`DoR: PASS`, a clean `kb_ticket_stale`, `kb_ticket_check` PASS, at most 10 ACs). The judgement boxes ("every AC is Given/When/Then and acceptance-testable", "backlog reviewed with the team") stay yours, and the agent never ticks an open question.

Change rules.

- **Done stories are frozen.** A `done` story keeps its backlog row. New or changed work is a **new row with the next free number**; numbers are never reused or renumbered.
- **Stories not yet done may be edited.** List each one in the row's `Affected`.
- **Decisions are superseded, not edited.** Never rewrite a `DECIDED` row. Add `D<n>` with `Supersedes D<k>` in its Decision cell and set `D<k>` to `SUPERSEDED`; `kb ticket check` then rejects any `[NEW: D<k>]`. A file that still cites the old row needs the `architecture` trigger.
- **Tickets follow their mission.** A ticket whose story a mission amendment lists in `Affected` becomes `draft` and drops out of the published plan until you amend it.

3.8 Publish the plan

Put a reviewed mission or ticket on the hub so developers, SAs and other teams find what is planned, not only what exists. You publish with `/kb-publish`, or by merging a change to `missions/` or `tickets/` into `main`.

```
/kb-publish          # or: kb publish
```

`kb publish` first runs `kb plan-ingest` and `kb build`, so what you review locally is what CI sends. After one explicit go from you it tags the commit `kb-publish/<timestamp>` and pushes the tag. The tag triggers `kb-publish.yml` (`kb plan-ingest`, `kb build`, `kb ci-publish`) and the intake opens a pull request on the hub; the content is live when that PR merges (at once when the hub's Registry has auto-merge on for your repo, otherwise when the hub maintainer merges it). The intake accepts only `refs/tags/kb-publish/*`. On the intake route `kb publish` refuses when `.kb/`, `missions/` or `tickets/` has uncommitted changes, because CI publishes the tagged commit. `.kb/<repo_id>-plan/` and `.kb/index.yaml` are generated and git-ignored: never commit them.

Merge is also publish. A push to `main` that touches `missions/` or `tickets/` runs `kb-publish-request.yml`, which tags the merged commit and dispatches the publish workflow (run it by hand to republish without a change; for another default branch, change `main` in two places in that file). Like `kb publish`, this needs `intake:` set, because `kb-publish.yml` runs `kb ci-publish`.

`kb plan-ingest` decides what ships, offline and without an LLM. Run it first to see:

Item	Published when
Mission	`kb mission lint` finds no structural error; its Definition of Ready is not a gate
Ticket	`kb ticket lint` finds no structural error, **and** every DoR box is ticked, **and** `kb mission next` would not demote it to `draft`

Whether a citation still resolves is not a publish gate (the PR gate checks it), and drafts never ship. Each item left out prints a line and never fails the command:

```
skipped: tickets/M-checkout/M-checkout-US3.md — Definition of Ready not fully ticked (6/8)
```

The main reasons: `lint error: <first error>` (run `kb mission lint` or `kb ticket lint` on the file); `Definition of Ready not fully ticked (<ticked>/<total>)` (finish the review and tick); `parent mission rev <M> affects this story` (amend the ticket, 3.7). For file-shape problems the message explains.

An item that stops qualifying drops out at the next publish; the hub's git history keeps earlier versions. `kb plan-ingest` exits `1` only when it cannot run, never for a skip.

On the hub. The plan is one document, `<repo_id>-plan`, next to `-code` and `-svc`: one group per mission with its published tickets, plus a `tickets` group for tickets with no published mission. Agents search it with `kb_search`; people read it under the web UI's **Ticket** menu (`/ui/tickets`: every BA repo, its missions and status counts; drill down to a mission's backlog and to a single ticket).

Status is derived and read-only: **done**, **ready**, **not ready** (no published ticket) or **blocked**. To change one, change the source. The tracker, not the hub, is the place for "in progress" and assignees. A Dev given only a ticket id fetches it from here (Developer Guide).

3.9 Keep citations current

A ticket written in March against a document amended in May must not silently start meaning something else.

```
kb resolve tickets/ABC-123.md          # ok | stale | broken, per ref
kb diff hr-handbook --against <rev>    # what actually changed
```

Status	Meaning	What to do
`ok`	The pinned content still matches what is published	Nothing
`stale`	The source was amended after you cited it	`kb diff`; decide whether the ACs need updating
`broken`	The reference no longer resolves	Re-ground and re-pin with `kb_context_new` or `kb context new`

Most staleness is harmless, such as a typo fix upstream; some is a requirement that no longer matches reality, and the judgement is yours. `kb ticket lint <file> --fail-on-stale` (or the `KB_FAIL_ON_STALE` variable for the CI

gate) makes staleness fail; `kb ticket stale [paths]` sweeps everything. The PR gate never sees a merged ticket again, so `kb-doctor.yml` sweeps daily and, while anything is stale or broken, keeps **one** open issue labelled `kb-stale-citations` (closed when everything is current):

Kind	Meaning	What to do
`kb-context`	A section the file cites in `## KB context` changed or no longer resolves	`kb diff`, then amend with `kb-context`, which re-pins it
`architecture`	A service, table, route, integration or command the file cites in `## Services & order` or `## Technical grounding` changed in the Dev repo's `-code` since it was grounded, or was removed	Amend with `architecture` so `sa-ticket-ground` re-grounds it; check the ACs that rest on it
`architecture`, `baseline not on hub`	The `Grounded on:` revision is not on the hub: the Dev PR that carried it never merged or was superseded	Wait for the Dev repo's `-code` to publish, then amend with `architecture`

Only the sections a file **cites** are compared, and not every row needs an amendment: if the requirement still holds, amend with the new pin and say so in the change-log row.

3.10 Stories across repos

A story that needs work in several dev repos is one **parent story** split into one **child ticket** per repo. The parent is the business story; each child is an ordinary ticket built in exactly one repo. Write parents only after the hub is upgraded to the release with parent stories (see Upgrade in the Hub Guide, 3.8): an older hub does not know the format.

The parent. `kb init` writes the shape to `docs/tickets/PARENT-TEMPLATE.md`. The file is `tickets/<folder>/<US-id>.md` with `> Kind: parent story` and `> Revision: <N>` in the header block, then `## User Story`, `## Split` and `## Change log` (the same six columns as a ticket's). A parent has no `## Acceptance Criteria` and no `## Definition of Ready`: every AC lives in exactly one child.

```
# Pay by card US-12
> Kind: parent story
> Revision: 1

## User Story
As a shopper, I want to pay by card, so that I can finish my order.

## Split
| Child | Repo | AC | Blocked by | Own fix | Why |
|---|---|---|---|---|---|
| US-12.payments-api | payments-api | AC1, AC2 | - | - | charges the card |
| US-12.web-app | web-app | AC3, AC4 | US-12.payments-api | - | the checkout form |

## Change log
| Rev | Date | Trigger | What changed | Affected | Review |
|---|---|---|---|---|---|
|  |  |  |  |  |  |
```

The header block of the child `tickets/<folder>/US-12.web-app.md`:

```
# Pay by card, checkout form US-12.web-app
> Revision: 1
> Parent story: US-12 @ rev 1
> Target repo: web-app
```

The rest of a child is an ordinary ticket (its own AC, DoR and grounding); `docs/tickets/PARENT-TEMPLATE.md` has the parent in full.

The children. A child id is `<parent-id>.<repo-id>` (`US-12.payments-api`, `US-12.web-app`); a later child for the same repo is `<parent-id>.<repo-id>.2`, then `.3`. A repo id that contains a dot cannot be a split target. A child repeats the parent's `## User Story` word for word and carries two header lines: `> Parent story: US-12 @ rev 1` and `> Target repo: payments-api`. A child of a mission's parent has no `> Parent mission:` line; it inherits the parent's. An AC that replaces one in a finished child is written `AC7 (replaces AC2)` in the split. `Blocked by` in the split names child ids from the same table; a child's own `- Blocked by:` counts only as those exact ids.

Changing a parent. Raise `> Revision:` and append a `## Change log` row (existing triggers only) naming the affected children in the same edit. A child that still says `@ rev <old>` and is named in a later row is demoted and leaves the hub until you amend it. A child also leaves the hub when its parent's mission amends the parent, until you amend the parent.

What lint checks. `kb ticket lint` on a parent checks the split (columns, every AC owned once, `Blocked by` ids exist in the split, no cycle) and the revision. On a child it checks the header against the parent in the same folder. A duplicate ticket id anywhere under `tickets/`, or a ticket id equal to a mission id, is an error. With `--hub`, an id already published by another BA repo, or a parent already done in a dev repo's `-svc`, is also an error. The CI gate also lints each parent in any folder a PR touched, so deleting a child fails the parent. `kb ticket lint` rejects `> Kind: parent bug` (not supported yet).

Publish fails closed. `kb plan-ingest` and `kb publish` publish a parent without a DoR. When any parent fails lint, nothing is published and the hub keeps its last good plan. Run `kb ticket lint` on the parent and fix the first error.

Status. The hub shows the parent as `in progress`, `blocked` or `done`, with `k/N done` (plus its mission's open blockers). Its page lists the children with repo and status and links each written one; a child's page links its parent and its siblings. A child waits on its split `Blocked by` ids; a child not written yet reads `not ready` (or `blocked` when the mission gates it). **Runners never take a parent**, and a claim on one answers 400 (`parent_story`): Dev runners work the children. A child not written yet is listed as to-draft on the hub; `/ba-ticket-author <parent-id>.<repo-id>` drafts it.

How the split is proposed. You do not decide the split alone. In `/ba-ticket-author`, step 1b "Scope by repo" reads which repos the need touches (`sa-ticket-ground --scope "<need>"`) and proposes one story or a parent with one child per repo, with the AC of each. Accept it and the skill writes the parent and the children. Reject it and the need stays one ticket; the skill writes `**Split decision:** <why>` under the parent's `## Change log` (a parent without that line gets a lint warning). To draft one child later, run `/ba-ticket-author <parent-id>.<repo-id>`: it reads the parent and writes only that child, without asking the split again.

`/ba-mission-plan` (step 5c, "Repos per story") can propose the same split while the mission is planned. That parent is written with `-` in the AC cell of every split row and no child files; `kb ticket lint` accepts it, the hub lists each child as to-draft, and the picker drafts them one by one. When you write a child, fill its AC into its split row. In full-auto mode `/ba-mission-plan` never splits: the split happens when the ticket is drafted, in step 1b, and an agent plus a fresh-context reviewer take the accept-or-reject decision.

Two lint warnings help here. A single ticket grounded on two or more repos warns `grounding touches several repos (<a>, <b>) — split it into a parent story (BA Guide 3.10) or keep one ticket`. A mission whose `## Services & order` names several repos is checked against each repo's `-code` document: write the ids as `<repo>:svc.<name>`.

---

4. Automation modes

Full-auto mode (optional). A switch per `ba` and `dev` repo that lets the two repos run on their own, on your machine, until every ticket on the hub is done. No step waits for a person: the agent decides each D-row and open question, answers the Dev's questions, ticks the Definition of Ready after its own review, merges and publishes. It writes down why in an `**Auto decision:**` line under the `## Change log`. It is built for **one BA repo and one Dev repo per hub**; `kb auto on` warns when the registry lists more than one Dev repo. It publishes only the plan and never edits the knowledge base. Nothing changes while the switch is off. The Dev side is in the Developer Guide.

4.1 Security: read this first

The permission deny list that `kb init` writes (6.5) is defence in depth, not a boundary. An unattended run merges its own pull requests and publishes to the hub with no human review. A ticket the agent misreads goes straight to `main` and to the hub. `kb auto on` prints this itself: "Auto mode merges pull requests as admin, with no review. Turn it on only for a repo where that is acceptable."

It merges with `kb pr merge` (GitHub REST), using the rights of your `gh` login; `--admin` is accepted for `gh` compatibility and changes nothing. If that login is a repo admin and branch protection does not include administrators, those merges bypass the protection. So, before you run it, the repo owner **must** enable GitHub branch protection on `main`:

- block force pushes and deletions;
- **include administrators**;
- keep the CI check as a **required status check**: the `lint` job of `kb-ticket-lint.yml`. Branch protection picks checks by job name. Without a required check, "green CI" is only a convention the agent follows;
- do **not** require approving reviews: the agent cannot approve its own PR, and the merge is then refused. If you require reviews anyway, exclude administrators from that rule and accept that the agent bypasses it on `main`.

The deny list also cannot see the current branch: the agent's working tree must never sit on `main` with commits on it. The skills branch first; the runner does not guard it. The runner never uses `--dangerously-skip-permissions` or `bypassPermissions`. The skills currently assume `main` as the default branch.

4.2 Set up once

1. Write the first mission yourself (`/ba-mission-plan`). The loop works the backlog it is given.
2. `intake:` is set in `.kb/config.yaml`, and the hub's Registry page has **auto-merge on** for this repo and for the Dev repo.
3. Re-run `kb init --kind ba`. It merges the permission allowlist into `.claude/settings.json` (your entries stay) and refreshes the skills.
4. Export `STRATA_KB_HUB_URL` and `STRATA_KB_HTTP_TOKEN` (the values behind `.mcp.json`), and have the `claude` CLI on the `PATH`, **logged in with a Claude subscription**. The runner never uses an API key and inherits whatever login `claude` has.

4.3 Run and stop

Task	Command	Result
Turn the switch on	`kb auto on`	prints the warnings, flips `auto.enabled`
Commit it	commit and push `.kb/config.yaml` and `.claude/settings.json` to the default branch	`kb auto run` refuses until you do: the Q&A worktree is checked out from the default branch
Start the loop	`kb auto run` (`--once`, `--model`, `--effort`), or `/kb-auto run`	`/kb-auto run` does all three steps and starts the loop detached; log in `.kb/auto/run.log`
See where it is	`kb auto status [--json]`, `/kb-auto status`	switch, task, attempts, questions waited on, stop reason
Stop it	`kb auto off`, `/kb-auto off`	stops after the current task
Wait for a result	`kb auto wait [--max 6600] [--interval 30]`	one line, used by the cloud watcher

`kb auto run` refuses, and prints every reason, when the switch is off, `.kb/config.yaml` or `.claude/settings.json` is untracked or dirty, `auto.enabled: true` is missing from the default branch's copy of the config, `intake:` is empty, or the hub registry cannot be read, does not list this repo, or lacks `auto_merge: true` for it. Exit codes: `0` done, `2` stopped, `1` refused or error, `130` interrupted.

4.4 What it does and when it stops

One fresh `claude -p` process per task. In order: a story held by an open D-row or blocking open question (`/ba-mission-plan --amend`), a `draft` ticket (`/ba-ticket-author --amend`), the next `to-draft` story (`/ba-ticket-author`), otherwise it waits `idle_interval` seconds. Each change goes through a branch, a PR, green CI and a merge, and the merge publishes (`kb-publish-request.yml`, 3.8). A second process, in its own worktree `.kb/auto/worktrees/qa`, answers the Dev's open questions with `/ba-answer-questions <ticket-id>` every `qa_interval` seconds. A ticket and its mission move together: a mission amend re-authors the not-done tickets it lists in `Affected`.

It stops after two failed attempts at one task (a red CI after two fix rounds stops it at once); `kb auto status` shows the reason, and `kb auto run` resumes after you fix it. A ticket whose Definition of Ready cannot be re-ticked is parked, not failed (its draft goes to `auto/qa-parked/<ticket>`); with only parked tickets left it stops with `parked: <tickets>`. A task still picked after 5 checks is skipped, and when every task left is skipped it stops. A failed `kb-publish` run is re-run once; when the re-run fails too, it stops with `publish failed for <tag>: <step>: <error>`. A usage-limit wait (15, 30, then 60 minutes) is not a failure. It finishes, exit `0`, when every story, unparented ticket and bug is `done` and no question is open. State and logs are in `.kb/auto/` (git-ignored).

A task process carries `STRATA_KB_AUTO_TASK=1`, under which `kb auto on`, `off` and `run` refuse (`status` still works), so a task can neither flip the switch nor start a second loop.

The runner's prompt asks each task to end its reply with one line: `AUTO-RESULT: done`, `AUTO-RESULT: blocked-qa Q<id>[,Q<id>...]` or `AUTO-RESULT: failed <reason>`, and nothing after it. A missing or malformed line counts as a failed attempt (`no result line`, `bad result line`). A skill you write for the runner must end the same way.

4.5 Claude Code cloud

`/kb-auto run` also works in a Claude Code cloud session (detected from `CLAUDE_CODE_REMOTE`; `STRATA_KB_CLOUD=1|0` forces it). Set `STRATA_KB_HUB_URL` and `STRATA_KB_HTTP_TOKEN` in the cloud environment. The hub is read live through its API and nothing is cloned; other hub-reading `kb` commands refuse and point at the `strata-kb` MCP tools. `kb pr` talks to the GitHub REST API instead of `gh pr`, because GraphQL is blocked there. Because nobody can click "trust this folder", `kb auto run` there marks the repo root and the Q&A worktree trusted in `~/.claude.json`; its other keys are kept and an unparseable file is left alone.

The session branch cannot push to the default branch. When the default branch's switch is off, `/kb-auto run` opens an `auto/switch` PR that flips only `auto.enabled`, waits for green checks and then **merges that PR itself, telling you first**, with `kb pr merge auto/switch --squash --admin --delete-branch`. The auto-mode settings must already be committed on the default branch. The same branch-protection advice as in 4.1 applies: that merge has no review either. `/kb-auto run` is re-runnable: it removes a leftover local `auto/switch` worktree and branch itself. A switch PR left open by a failed run (red checks, merge error) and its remote `auto/switch` branch must be closed and deleted on GitHub by hand first; the skill stops and says so, and never force-pushes. **The session must stay open** for the whole run. Not yet verified on real cloud: a session lasting over three hours, pushing `auto/*` branches, a merge past a required review, and `gh run` over REST.

---

5. Troubleshooting

6. Appendix

6.1 Config keys and environment variables

Key or variable	Meaning
`kind`, `hub`, `repo_id`, `intake`	see 2.2. Never put a token in `hub:` or `intake:`
`auto.enabled`, `auto.qa_interval`, `auto.idle_interval`, `auto.hotfix_base`	auto mode; by default `qa_interval` is `300` and `idle_interval` `600` seconds; `hotfix_base` empty = default branch (the skills currently assume `main`). The scaffold only comments the block; `kb auto on` writes the live one
`STRATA_KB_HUB`	overrides `hub:`; carries a credentialed URL in CI (an Actions variable)
`STRATA_KB_HUB_URL`, `STRATA_KB_HTTP_TOKEN`	the hub's HTTP base URL and bearer token: MCP and the auto runner
`KB_HUB_TOKEN`, `KB_FAIL_ON_STALE`	Actions secret (private hub only) and Actions variable of `kb-ticket-lint.yml` and `kb-doctor.yml`
`STRATA_KB_AUTO_TASK`	set by the runner on task processes; `kb auto on`, `off` and `run` refuse under it
`STRATA_KB_CLOUD`, `CLAUDE_CODE_REMOTE`	force or detect Claude Code cloud

Exit codes: `0` ok, `1` error, `2` stale citation (or a usage error); `kb auto` as in 4.3. The skills call the `strata-kb` MCP tools; the full list is in the hub guide.

6.2 What `kb init` writes, and what is git-ignored

Path	Holds	Committed
`tickets/`, `missions/`	your tickets and mission plans	yes
`docs/ac-quality.local.md`, `docs/review-rubric.local.md`	your overrides; created once, never refreshed	yes
`.kb/*-plan/`, `.kb/index.yaml`	generated by `kb plan-ingest`	no
`.kb/auto/`	the auto runner's state (`kb init` adds it to `.gitignore`)	no
`.env`	the MCP credentials (`kb mcp-setup` makes git ignore it; `kb init` does not)	no

Re-running `kb init --kind ba` creates missing files, refreshes scaffold files that differ from the template and appends the `.kb/*-plan/`, `.kb/index.yaml` and `.kb/auto/` ignore lines to `.gitignore`, never removing one. `.kb/config.yaml`, `.kb/index.yaml` and `.claude/settings.json` are protected: a plain run leaves the first two alone and only merges into the third (your entries stay). **`--force` replaces all three outright**, including `kind`, `repo_id` and your whole `.claude/settings.json`. Nothing under `tickets/` or `missions/` is touched. Never commit `.kb-qa/`: it is the hub's Q&A store, git-ignored on the hub.

6.3 What the gate checks

Errors (the gate fails): a missing required section (Summary, User Story, Background / Business context, Acceptance Criteria, Use cases, Business flow, KB context, Definition of Ready); a heading twice; more than 10 acceptance criteria or more than one user story (split the ticket); a `## KB context` ref that does not resolve at its pinned commit; an inline `[doc-id §section]` citation not backed by a pinned ref; a change log that does not match the revision. **Warnings (your judgement):** a criterion touching a standard with no citation; a stale ref; a pinned ref the body never cites; a missing `> Revision:` line or `## Review record`; an AC that prescribes a shell command or is not Given/When/Then; a missing or empty extended section the story's words call for.

6.4 Code knowledge in tickets

Besides domain documents, the hub holds two documents per product repository, published by that repository's own workflow: `<repo>-code` (generated structure: `svc.*`, `db.*`, `api.*`, `int.*`, `cmd.*`, `route.*`, `page.*`, `job.*`, `sym.*`, `struct.tree`, `struct.files.*`, `struct.coverage`) and `<repo>-svc` (curated responsibility). The BA skills never read either themselves: where a ticket needs a service, table, route or file name they write `%%TODO: verify against codebase%%` with an owned open question, and `/sa-ticket-ground` answers it from `<repo>-code`. Everything it writes is checked by `kb ticket check`; what the document cannot prove goes under `Open decisions` for the developer. A section reference is spelled `<repo>-code §svc.<name>`.

> **One caution.** `<repo>-svc` grounds a diagram. It is never a substitute for a domain citation in an acceptance criterion. A code, format, enum or threshold that encodes a standard must come from a pinned domain section.

`struct.coverage` tells the SA how far to trust an absence, per area: `complete`, `partial` or `not scanned` (for example the Dev repo lacks `strata-kb[code]`). Absent from a `complete` scope means not built: the SA writes `<thing> in <path> [NEW: D<n>]` (a file is its own `Files:` entry, `<path> [NEW: D<n>]`). Absent from a `partial` or `not scanned` scope means unknown, so it goes under `Open decisions` and the gate fails a `[NEW]` there. A job is never `complete` (`jobs outside framework schedulers are not detectable`). Detail: Developer Guide.

6.5 Auto-mode permissions

`kb init --kind ba` merges its permissions into your `.claude/settings.json`. **Allowed:** everyday `git` (add, commit, status, diff, log, fetch, pull, switch to `auto/*`, `fix/*`, `hotfix/*`, branch, worktree, merge, `push -u origin HEAD`), `gh pr` and `gh run`, `Read`/`Edit`/`Write`, the `strata-kb` MCP tools and many `kb` subcommands. **Denied:** any `git push` that names `main` or `master`, force-push, `+` and `:` refspecs and `--delete`; `git checkout`, `switch` and `branch` forms that move onto, create, rename or delete `main` or `master` (a branch id that merely contains the word, like `auto/M-domain-US1`, is fine); `git reset --hard`; `gh repo delete`, `gh run delete`; `kb init`; `rm -rf /*`; editing `.claude/settings.json`, `.claude/settings.local.json` and `.git/**`. The `Stop` and `SubagentStop` hooks run `kb usage ingest-transcript --hook-stdin`; `kb usage report` turns that ledger into a cost report (`.kb/usage/report.html`).

Cost reports. The hook adds about 0.6 s per turn (the cost of starting `kb`). `kb usage report` writes `.kb/usage/report.html`, per ticket, phase and model; with `--ticket <id> --md` it prints a Markdown table. A failing hook is silent by design (a `Stop` hook cannot block a turn from ending), so if `.kb/usage/*.jsonl` is not growing, read `.kb/usage/ingest-errors.log`: it is the only place a failure inside `kb` shows. Re-ingesting an ended transcript is safe, because a call already in the ledger is skipped: `kb usage ingest-transcript <path>` backfills

one. Run it only on a session that has ended: a call still streaming would be stored with a partial output count, and a stored row never changes. `report.html` is regenerated from the ledger on every run, so the ledger is the record, not the report. **Model tiering** is a recommendation that nothing in `kb` enforces. Model prices come from `usage-prices.yaml` (shipped, overridable per model at `.kb/usage-prices.yaml`), which carries an `effective_date`. A model with no row is reported `unpriced`, never guessed, and fast-mode calls are priced at the standard rate and flagged, so a total that contains them is a lower bound. Switch model at phase boundaries, never inside a phase: a switch discards the prompt cache and pays a fresh cache write, so a few cheap-model turns inside a strong-model session can cost **more** than not tiering. Check the result with `kb usage report`, which breaks spending down by model.

6.6 Hub Q&A rules that touch you

Questions and answers live in the hub's Q&A store, written only through `kb_question_ask` (Dev) and `kb_question_answer` (BA). `asked_by` and `answered_by` are names the caller types (the skills pass `git config user.name`): tracking, not authentication. Limits: 4000 characters for a question or answer, 1000 for context, 100 for names, at most 50 changed sections per answer and 50 open questions per ticket.